



DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE FOR ROBOTICS 2

Agricultural Robotics

Energy Constrained Field Operations Report

Author :

Bahri Riadh

Professor :

Fulvio Mastrogiovanni

Student ID :

S8335614

Tutor :

Omar Kashmar

27 juin 2026



INTRODUCTION

In agricultural robotics, long-horizon autonomy requires precise resource management. An autonomous robot operating in an open field must accomplish its assigned tasks without depleting its finite energy reserves. This project explores the critical intersection of route planning and energy constraints using automated planning techniques.

We model a mobile robot tasked with completing agricultural operations across a field. Because every movement and action consumes energy, the planner must find action sequences that satisfy the farming goals while respecting strict battery limits, including autonomous returns to a docking station to recharge when necessary.

To solve this combined routing and resource management problem, this project utilizes the ENHSP (Expressive Numeric Heuristic Search Planner) to develop and evaluate two distinct automated planning models :

1. The Discrete Numeric Model (Q1) : Abstraction of energy as discrete, instantaneous step costs using classical PDDL numeric fluents.
2. The Continuous Time Model (Q2) : A highly realistic PDDL+ formulation where energy discharge and navigation are treated as continuous temporal processes.

The objective is to synthesize plans that satisfy the farming goals while strictly adhering to the numeric energy boundaries, forcing the planner to discover survival behaviors such as mid-mission recharging and terrain-aware routing.

The Physical Environment

The agricultural field is modeled as a connected graph : nodes are locations and edges are traversable paths, each carrying a cost paid out of the robot's finite battery. The same field underlies both models ; what differs is how an edge's cost is expressed , a single discrete number in Q1, a continuous discharge rate in Q2.

1.1 Field Topology

The environment consists of :

- **The Dock (D)** : Robot start position and the only location where recharge is allowed.
- **The Plots (P1–P4)** : Agricultural zones where specific operations must be executed.
- **Waypoint (W1)** : An alternative longer-but-cheaper route node, used only in Q2 to set up a routing dilemma ; the Q1 problems use the dock–plot core graph without it.

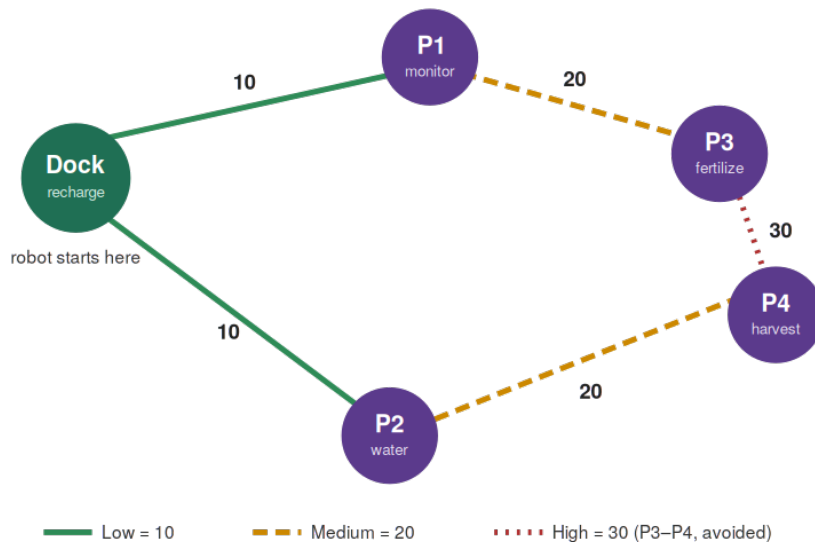


FIGURE 1.1 – Q1 discrete cost graph. Edge labels are the move-cost values ; the P3–P4 high-cost edge is a high-cost link that no plan uses. W1 does not appear in Q1.

1.2 Traversal Costs and Payload

Move energy depends on two factors : the base difficulty of the edge and the dynamic payload weight carried by the robot.

1.2.1 Q1 — Discrete cost tiers

Each edge has a fixed numeric move-cost in one of three tiers. These are numeric weights, not symbolic terrain types , the Q1 domain reasons purely over the number.

Cost Tier	move-cost	Edges in this tier
Low	10	Dock-plot links : D $\leftrightarrow$ P1, D $\leftrightarrow$ P2.
Medium	20	Inter-plot links : P1 $\leftrightarrow$ P3, P2 $\leftrightarrow$ P4.
High	30	The high-cost link P3 $\leftrightarrow$ P4, unused by any plan.

Move energy is the edge cost plus the current payload : $(+ (\text{move-cost } ?\text{from } ?\text{to}) (\text{load } ?r))$, so a heavier robot pays more on every edge.

1.2.2 Q2 : Terrain types as discharge rates

The continuous model replaces the single cost with a *discharge rate* (energy per second) active for the whole move duration. Two terrain predicates, (`flat` ?l1 ?l2) and (`muddy` ?l1 ?l2), select the rate :

Terrain (Q2)	Discharge rate	Description
Flat	$5 + \text{load}$	Smooth paths; baseline rate plus payload penalty.
Muddy	$15 + \text{load}$	Taxing paths; triple the base rate, used to stress the energy budget.

Total move energy is $\Delta t \times (\text{rate}_{\text{terrain}} + \text{load})$, so both move *duration* and *load* scale the cost continuously. The Q1→Q2 terrain mapping is a per-problem choice : a medium-cost Q1 edge may be declared muddy in Q2 to stress the energy budget.

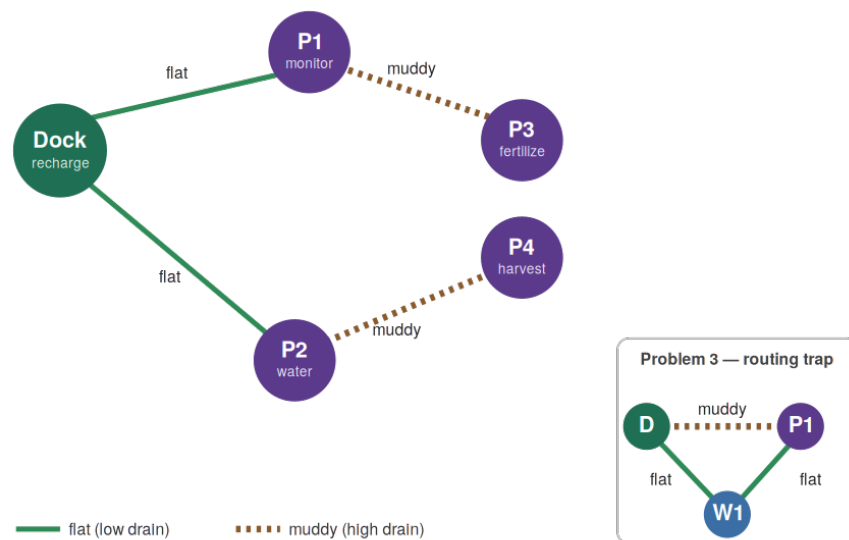


FIGURE 1.2 – Q2 terrain view. flat/muddy predicates set the discharge rate, assigned per problem. The W1 detour (inset) is used only in Problem 3.

1.3 Agricultural Operations

Each plot requires a specific task. Heavier mechanical tasks (harvesting) cost more than lighter ones (monitoring), and each operation changes the payload, which feeds back into movement cost in *both* models.

TABLE 1.1 – Agricultural operations and their dynamic payload effects

Operation	Assigned Plot	Energy Cost	Payload Effect
Monitor	P1	5	Unchanged
Water	P2	15	Decreases load by 5 (lighter return)
Fertilize	P3	20	Decreases load by 5 (lighter return)
Harvest	P4	25	Increases load by 5 (heavier return)

These are the discrete op-cost values used in Q1 ; in Q2 each operation runs for a fixed *duration* with continuous drain instead. The payload effect is identical in both models. With a maximum battery capacity of 100 units and an initial payload of 5, edge costs, operation costs, and the load penalty together define each mission's total energy budget.

Question 1 : Discrete Numeric Model

We model the environment using classical PDDL with numeric fluents. The core deliverable of Q1 is showing that energy can be treated as a hard numeric resource depending on terrain, operations, and the dynamic load carried by the robot.

2.1 The PDDL Abstraction

Energy is a discrete numeric fluent : we avoid boolean battery flags and instead track (battery ?r) and (capacity ?r).

2.1.1 Predicates and Functions :

- **Position** : (robot-at ?r ?l) (we use robot-at rather than at, which ENHSP reserves as a temporal keyword).
- **Operation State** : (needs-monitor ?l) is consumed and replaced by (monitored ?l).
- **Numeric Costs** : (move-cost ?from ?to) encodes terrain difficulty; (op-cost-*) defines operation weights.

2.1.2 Action Dynamics and Energy Consumption :

Each action has a strict energy precondition ($\geq$ (battery ?r) cost) and a decrease effect. The recharge action, restricted to the Dock, uses assign to instantly reset the battery to capacity.

2.1.3 Dynamic Load Integration :

The (load ?r) fluent couples operations to movement cost :

- **Movement** : Move energy is terrain cost plus current load : (+ (move-cost ?from ?to) (load ?r)).
- **Operations** : water and fertilize decrease load by 5 (cheaper subsequent travel); harvest increases it by 5 (costlier return).

2.2 Results and Planner Outputs

2.2.1 Problem 1 : Feasible Without Recharge

Goal : (monitored P1) $\wedge$ (watered P2) $\wedge$ (robot-at r1 D). Two operations on plots close to the dock, starting with payload load = 5 for the watering task.

Conclusion for P1 : On the return move the robot pays only 10 instead of 15, because it dropped its payload during watering. It finishes with 25 units to spare and ENHSP reports just 1 dead-end : comfortably feasible on a single charge. Full plan : plans/Q1_plan1.txt.

2.2.2 Problem 2 : Recharge Required

Goal : $(\text{fertilized } P3) \wedge (\text{harvested } P4) \wedge (\text{robot-at } r1 \text{ } D)$. The robot must service plots far from the dock, where terrain cost and dynamic load combine into a tightly constrained problem.

Planner Output :

TABLE 2.1 – Energy and load execution trace for Problem 2

Found Plan:

```
0.0: (move r1 D P1)
1.0: (move r1 P1 P3)
2.0: (fertilize r1 P3)
3.0: (move r1 P3 P1)
4.0: (move r1 P1 D)
5.0: (recharge r1 D)
6.0: (move r1 D P2)
7.0: (move r1 P2 P4)
8.0: (harvest r1 P4)
9.0: (move r1 P4 P2)
10.0: (move r1 P2 D)
```

Step	Action	Energy Cost Calculation	Load After	Battery After
–	Initial State	—	5	100
0	move r1 D → P1	Terrain(10) + Load(5) = 15	5	85
1	move r1 P1 → P3	Terrain(20) + Load(5) = 25	5	60
2	fertilize r1 P3	Operation Cost = 20	0	40
3	move r1 P3 → P1	Terrain(20) + Load(0) = 20	0	20
4	move r1 P1 → D	Terrain(10) + Load(0) = 10	0	10
5	recharge r1 D	Full Replenishment	0	100
6	move r1 D → P2	Terrain(10) + Load(0) = 10	0	90
7	move r1 P2 → P4	Terrain(20) + Load(0) = 20	0	70
8	harvest r1 P4	Operation Cost = 25	5	45
9	move r1 P4 → P2	Terrain(20) + Load(5) = 25	5	20
10	move r1 P2 → D	Terrain(10) + Load(5) = 15	5	5

```
Plan-Length:11
Metric (Search):11.0
Expanded Nodes:28
States Evaluated:32
Dead-Ends:20
```

Conclusion for P2 : The problem shows in the statistics : 20 dead-ends, driven by the heavy load making long trips cost more, so a single charge is insufficient. The robot travels to P4 empty, harvests (load +5), and returns carrying the yield over the medium-cost edges, arriving at the dock with exactly 5 units left. This shows how routing and energy affect each other, together with operation sequencing and resource management.

2.2.3 Validation with VAL

Both plans were validated with VAL. For **Problem 2**, VAL confirms the discrete recharge at step 5, resetting the battery from its pre-recharge low of 10 up to the full capacity of 100 :

```
Checking next happening (time 5)
Updating (battery r1) (10) by 100 assignment
...
Plan executed successfully - checking goal
Plan valid
```

The trace confirms the energy budget leaves little margin : a pre-recharge low of 10 units (step 4), reset to 100, and a final arrival with exactly 5 units (step 10). VAL labels the recharge as an assignment rather than an increase, verifying the discrete Q1 semantics work as designed.

Question 2 : Continuous Time Model

3.1 PDDL+ Domain Architecture

In Question 2, we upgrade our discrete foundation to a continuous PDDL+ architecture. To make the simulation behave like a real physical robot, we swapped out the static, step-by-step actions for dynamic, continuous time. This shift comes down to three main changes :

3.1.1 Handling Time : The "Start & Stop" Mechanism

Standard PDDL actions happen instantly. To simulate real-world time without standard durative actions, we split every operation (like moving or watering) into two distinct steps :

- **Start** : The robot begins the task, and a local timer (*time-spent*) is reset to 0.
- **Stop** : The task only finishes when *time-spent* exactly matches the required duration. This strict rule forces the planner to respect real time and prevents it from cutting corners.

3.1.2 Real-Time Battery Drain : Continuous Processes

In the discrete model (Q1), energy was subtracted as a fixed penalty. In Q2, energy drains constantly in the background while the robot is moving or working. We used Processes to model this continuous drain. The longer the task takes, the more battery is lost. The speed of this drain dynamically changes based on the terrain and how heavy the robot's load is :

$$E_{lost} = \Delta t \times (Rate_{terrain} + Load_{current})$$

Here $Rate_{terrain}$ is 5 per second on flat edges and 15 per second on muddy edges. Operating tasks (monitor, water, etc.) drain at a fixed 5 per second, independent of load.

3.1.3 Automatic Failsafes : Autonomous Events

To stop the planner from generating impossible solutions (like driving with a dead battery or overcharging past 100%), we implemented Events. These act as invisible, automatic sensors :

- **Critical Depletion** : If the continuous battery drain hits 0.0, this event triggers instantly, breaking the safety goal and failing the mission.
- **Overcharge Prevention** : When the robot is charging at the dock, this event automatically cuts the power the exact millisecond the battery reaches its maximum capacity.

3.2 Results and Planner Outputs

All three problems carry the safety goal (`not (depleted r1)`) and minimize total time. The trace tables below give the battery level sampled at the end of every move or operation ; the full ENHSP plans are in the repository `plans/` folder. Problem 2's raw output is shown once as a representative sample of the start/stop/wait structure.

3.2.1 Problem 1 : Continuous Baseline (All Flat)

Goal : (monitored P1) $\wedge$ (watered P2) $\wedge$ (robot-at r1 D) $\wedge$ (not (depleted r1)).
Every edge is flat, so the robot drains at the baseline rate plus its load. Full plan : plans/Q2_plan1.txt.

TABLE 3.1 – Continuous energy and load trace for Problem 1

End t	Segment	Drain	Load	Battery
0	Initial State	—	5	100
1	move D $\rightarrow$ P1 (flat)	$1 \times (5+5)$	5	90
2	monitor P1	1×5	5	85
3	move P1 $\rightarrow$ D (flat)	$1 \times (5+5)$	5	75
4	move D $\rightarrow$ P2 (flat)	$1 \times (5+5)$	5	65
7	water P2 (3s)	3×5	0	50
8	move P2 $\rightarrow$ D (flat)	$1 \times (5+0)$	0	45

Conclusion for P1 : The robot finishes the mission at $t = 8$ with 45 units left and is never depleted. The last move costs only 5 instead of 10, because watering dropped the load to 0 , the same load effect seen in Q1, now applied continuously over the move duration.

3.2.2 Problem 2 : Muddy Mission with Recharge

Goal : (fertilized P3) $\wedge$ (harvested P4) $\wedge$ (robot-at r1 D) $\wedge$ (not (depleted r1)). The plot links P1–P3 and P2–P4 are muddy (15/s), so the robot must recharge between the two halves of the mission.

Planner Output :

```
0: (start-move r1 D P1)
1.0: (stop-move r1 D P1)
1.0: (start-move r1 P1 P3)
2.0: (stop-move r1 P1 P3)
2.0: (start-fertilize r1 P3)
6.0: (stop-fertilize r1 P3)
6.0: (start-move r1 P3 P1)
7.0: (stop-move r1 P3 P1)
7.0: (start-move r1 P1 D)
8.0: (stop-move r1 P1 D)
8.0: (start-recharging r1 D)
10.0: (stop-recharging r1)
10.0: (start-move r1 D P2)
11.0: (stop-move r1 D P2)
11.0: (start-move r1 P2 P4)
12.0: (stop-move r1 P2 P4)
12.0: (start-harvest r1 P4)
17.0: (stop-harvest r1 P4)
17.0: (start-move r1 P4 P2)
18.0: (stop-move r1 P4 P2)
18.0: (start-move r1 P2 D)
19.0: (stop-move r1 P2 D)
```

TABLE 3.2 – Continuous energy and load trace for Problem 2

End t	Segment	Drain	Load	Battery
0	Initial State	—	5	100
1	move D $\rightarrow$ P1 (flat)	$1 \times (5+5)$	5	90
2	move P1 $\rightarrow$ P3 (muddy)	$1 \times (15+5)$	5	70
6	fertilize P3 (4s)	4×5	0	50
7	move P3 $\rightarrow$ P1 (muddy)	$1 \times (15+0)$	0	35
8	move P1 $\rightarrow$ D (flat)	$1 \times (5+0)$	0	30
10	recharge (2s)	$+2 \times 25$	0	80
11	move D $\rightarrow$ P2 (flat)	$1 \times (5+0)$	0	75
12	move P2 $\rightarrow$ P4 (muddy)	$1 \times (15+0)$	0	60
17	harvest P4 (5s)	5×5	5	35
18	move P4 $\rightarrow$ P2 (muddy)	$1 \times (15+5)$	5	15
19	move P2 $\rightarrow$ D (flat)	$1 \times (5+5)$	5	5

Conclusion for P2 : At 30 units after the first half, the planner inserts a 2-second recharge that the recharge-battery process lifts back to 80 ($30 + 2 \times 25$); the robot then harvests and returns at $t = 19$ with 5 units left. Recharge is a timed process, not an instant reset : the robot must *wait* to gain energy, and waiting past the 100 cap is stopped by the overcharge event.

3.2.3 Problem 3 : Routing Under Continuous Drain

Goal : $(\text{harvested } P1) \wedge (\text{robot-at } r1 \text{ } D) \wedge (\text{not } (\text{depleted } r1))$. The dock reaches P1 either by a short muddy edge (2s direct) or a longer flat detour through W1. The muddy edge is faster but drains at the high rate; the detour is slower but cheaper per second. Full plan : plans/Q2_plan3.txt.

TABLE 3.3 – Continuous energy and load trace for Problem 3

End t	Segment	Drain	Load	Battery
0	Initial State	—	5	100
2	move $D \rightarrow P1$ (muddy, 2s)	$2 \times (15+5)$	5	60
7	harvest P1 (5s)	5×5	10	35
8	move $P1 \rightarrow W1$ (flat)	$1 \times (5+10)$	10	20
9	move $W1 \rightarrow D$ (flat)	$1 \times (5+10)$	10	5

Conclusion for P3 : Minimizing total time, the planner takes the muddy edge *into* P1 while the robot is light, then the flat W1 detour *out*, once the harvested load makes muddy travel too expensive. It survives with 5 units to spare. Unlike Q1, the route is not chosen on a fixed distance cost : the same edge costs differently depending on the time spent on it and the load carried at that moment.

3.3 Discussion

3.3.1 Discrete vs. Continuous Modelling

Q1 treats energy as a number that drops in one jump per action. It is simple, fast to solve, and easy to check by hand, but recharging is instantaneous and a move costs the same whatever its duration. Q2 drains energy over time through a process, so duration matters and recharging takes real seconds : in Problem 2 the robot must wait two seconds at the dock to go from 30 back to 80 before it can continue. Q1 is enough when only the order of actions matters ; Q2 is needed when how long things take is part of the problem.

3.3.2 Routing and Resource Management Interact

The cheapest route and the survivable route are not the same. In Q1 Problem 2 the shortest sequence runs out of energy, so the planner has to add a recharge, which changes the route itself. In Q2 Problem 3 the robot crosses the muddy edge while light but takes the flat detour once the harvested load makes that edge too expensive the route depends on the resource state, not on distance alone. Energy and routing are one combined problem.

3.3.3 Energy in Long-Horizon Autonomy

For a robot running on its own, energy decides whether a mission is possible at all. Our margins are thin by design : Q1 Problem 2 and Q2 Problem 3 both end at exactly 5 units. The recharge action lets the robot extend a mission beyond one battery, and the depletion event makes hitting zero a hard failure to avoid. Together they produce the survival behaviour the assignment asks for returning to the dock when needed and refusing routes it cannot finish.